Jaypee Institute of Information Technology, Noida

CN Lab

Assignment 9

Introduction to Secure Socket Programming

1. Secure Socket Programming

In this lab, you will learn the basics of secure socket programming for TCP connections in Python. You will learn how to create a socket, establish a connection between a client and a server, and securely exchange data using encryption and decryption. You will use the Fernet module from the Python cryptography library to encrypt messages before transmission and decrypt them at the receiving end.

You will develop secure client-server applications:

- That establish TCP connections between clients and servers.
- That encrypt messages before sending them over the network and decrypt them at the receiving end.
- That use a shared secret key for secure communication.
- That support operations such as message exchange, calculator operations, and string manipulation.
- That can handle multiple clients simultaneously using multithreading.
- That encrypt responses before sending them back to the client, ensuring secure data exchange.

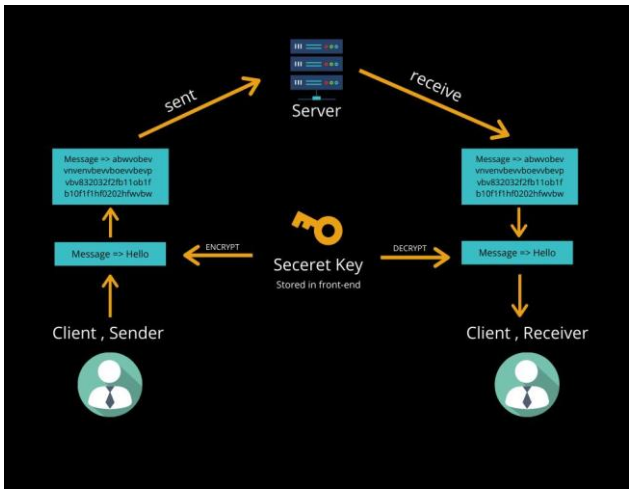


Figure 1: Secure Client–Server Communication Using Encryption and Decryption.

Examples:

Here we can use **Fernet** from the cryptography library.

Install it with:

pip install cryptography

First generate a key:

```
from cryptography.fernet import Fernet

key = Fernet.generate_key()

with open("secret.key", "wb") as f:
    f.write(key)

print("Key generated successfully")
```

Run this once. Both client and server need access to secret.key.

Server

```
import socket
from cryptography.fernet import Fernet

HOST = "127.0.0.1"
PORT = 5000

with open("secret.key", "rb") as f:
    key = f.read()

cipher = Fernet(key)

server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
server.bind((HOST, PORT))
server.listen(1)

print("Waiting for client...")

conn, addr = server.accept()
print("Connected:", addr)

encrypted_message = conn.recv(4096)

message = cipher.decrypt(encrypted_message).decode()

print("Decrypted message:", message)

conn.send("Message received securely".encode())
```

```
conn.close()
server.close()
```

Client

```
import socket
from cryptography.fernet import Fernet

HOST = "127.0.0.1"
PORT = 5000

with open("secret.key", "rb") as f:
    key = f.read()

cipher = Fernet(key)

client = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
client.connect((HOST, PORT))

message = input("Enter message: ")

encrypted_message = cipher.encrypt(message.encode())

client.send(encrypted_message)

response = client.recv(4096).decode()
print("Server:", response)

client.close()
```

Q1. Write a TCP client-server program in which the client encrypts a message before sending it to the server. The server should decrypt the message and display the original message.

Q2. Develop a multithreaded TCP server that can handle multiple clients simultaneously. Messages sent by each client should be encrypted before transmission and decrypted by the server.

Q3. Create a TCP client-server calculator where the client sends two numbers and an operation (+, -, *, /) to the server. Encrypt the data before sending it, and return the encrypted result to the client.

Q4. Develop a client-server application where the client sends an encrypted string to the server. After decryption, the server should perform operations such as:

- Count characters
- Convert to uppercase
- Reverse the string

The result should be encrypted before being sent back.

Q5. Create a multithreaded TCP server that accepts multiple clients. Each client can choose one of the following operations:

- Send a message
- Perform a calculator operation
- Request a string operation